

Hospital Medical Database

PostgreSQL

Docker

CS377 Ursinus College

Course: CS377.

Author: Karthik Reddy Akkala.

Date: May 2, 2026.

Introduction:

This project is for implementing a PostgreSQL based hospital management database for managing patients, medical test, and all other records.

This has been containerized with docker for easy setup and working efficiently across different environments.

Tools:

- PostgreSQL
 - Docker
-

Project Structure:

```
CS377-A_hospital_medicaldb/  
├── README.md  
├── database  
│   ├── backups  
│   │   └── backup.sql  
│   ├── init.  
│   │   ├── schema.sql  
│   │   └── seed.sql  
├── docker-compose.yml  
├── queries  
│   ├── function.sql  
│   ├── procedures.sql  
│   └── queries.sql  
└── scripts  
    └── backup_db.sh
```

Setting Up Database:

Start & Connect To Database:

```
git clone https://github.com/karthik-bit1/CS377-A_hospital_medicaldb.git
```

```
cd CS377-A_hospital_medicaldb
```

```
docker compose up -d
```

```
docker exec -it hospital_postgres psql -U postgres -d hospital_meddb
```

Stop Database:

```
docker compose down
```

Reset Database:

```
docker compose down -v
docker compose up
```

Backup Database:

Running Backups:

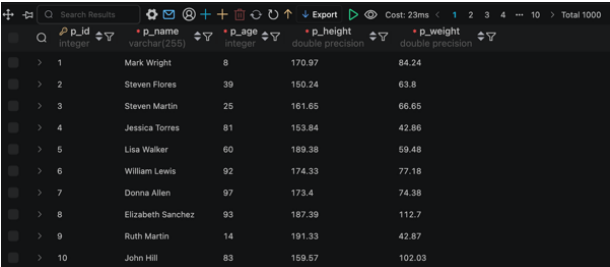
```
./scripts/backup_db.sh
```

Backups saved in:

```
CS377-A_hospital_medicaldb/
├── README.md
├── database
│   └── backups
│       └── backup.sql
```

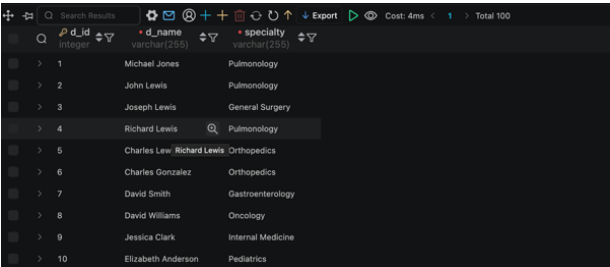
Tables In The Database:

- **patient:** stores patient information



p_id	p_name	p_age	p_height	p_weight
integer	varchar(255)	integer	double precision	double precision
1	Mark Wright	8	170.97	84.24
2	Steven Flores	39	150.24	63.8
3	Steven Martin	25	161.65	66.65
4	Jessica Torres	81	153.84	42.86
5	Lisa Walker	60	189.38	69.48
6	William Lewis	92	174.33	77.18
7	Donna Allen	97	173.4	74.38
8	Elizabeth Sanchez	93	187.39	112.7
9	Ruth Martin	14	191.33	42.87
10	John Hill	83	159.57	102.03

- **doctor:** stores doctor details



d_id	d_name	specialty
integer	varchar(255)	varchar(255)
1	Michael Jones	Pulmonology
2	John Lewis	Pulmonology
3	Joseph Lewis	General Surgery
4	Richard Lewis	Pulmonology
5	Charles Lew	Richard Lewis Orthopedics
6	Charles Gonzalez	Orthopedics
7	David Smith	Gastroenterology
8	David Williams	Oncology
9	Jessica Clark	Internal Medicine
10	Elizabeth Anderson	Pediatrics

- **appointments:** tracks patients appointments with their doctor

A screenshot of a database query result for the 'ap' table. The table has columns: ap_id (integer), ap_dat (date), ap_time (time without time zone), p_id (integer), and d_id (integer). The results show 10 rows of data with various dates and times.

	ap_id integer	ap_dat date	ap_time time without time zone	p_id integer	d_id integer
> 1	1	2023-05-01	06:45:00	287	93
> 2	2	2026-10-03	11:00:00	566	38
> 3	3	2023-10-15	11:30:00	856	93
> 4	4	2024-06-29	14:45:00	179	23
> 5	5	2023-08-06	09:15:00	942	39
> 6	6	2024-09-06	16:45:00	32	40
> 7	7	2024-05-06	10:30:00	532	89
> 8	8	2026-07-30	11:30:00	965	7
> 9	9	2025-02-07	13:00:00	757	28
> 10	10	2025-06-29	10:45:00	215	88

- **med_test**: stores medical test results

A screenshot of a database query result for the 'mt' table. The table has columns: mt_id (integer), mt_name (varchar(255)), mt_result (varchar(255)), mt_dat (date), p_id (integer), and d_id (integer). The results show 10 rows of data for various medical tests.

	mt_id integer	mt_name varchar(255)	mt_result varchar(255)	mt_dat date	p_id integer	d_id integer
> 1	1	Blood Test	negative	2025-06-04	244	11
> 2	2	Allergy Test	positive	2025-07-28	682	66
> 3	3	MRI Scan	abnormal	2024-06-19	244	20
> 4	4	MRI Scan	positive	2023-10-04	700	4
> 5	5	COVID-19 Test	pending	2025-12-06	142	31
> 6	6	CT Scan	positive	2023-08-12	528	81
> 7	7	Hemoglobin Test	normal	2025-05-31	607	43
> 8	8	Cholesterol Test	pending	2025-09-14	530	75
> 9	9	Urine Test	pending	2026-06-20	424	26
> 10	10	Hemoglobin Test	positive	2025-05-09	516	7

- **medicine_price**: stores medicine pricing

A screenshot of a database query result for the 'mp' table. The table has columns: mp_id (integer), med_name (varchar(255)), and price (double precision). The results show 10 rows of data for various medicines.

	mp_id integer	med_name varchar(255)	price double precision
> 1	1	Dexpri Max	282.34
> 2	2	Hydroclin Tab	259.74
> 3	3	Roxidine Forte	227.99
> 4	4	Dolotidine Cap	113.1
> 5	5	Hydroclol Max	370.91
> 6	6	Cipromab XR	255.31
> 7	7	Trixan Max	230.23
> 8	8	Zyloprazole Cream	266.78
> 9	9	Parvir Lite	137.66
> 10	10	Roxisartan DS	357.84

- **prescriptions**: records prescriptions issued

A screenshot of a database query result for the 'pr' table. The table has columns: pr_id (integer), ap_id (integer), p_id (integer), d_id (integer), and pr_date (date). The results show 10 rows of data for various prescriptions.

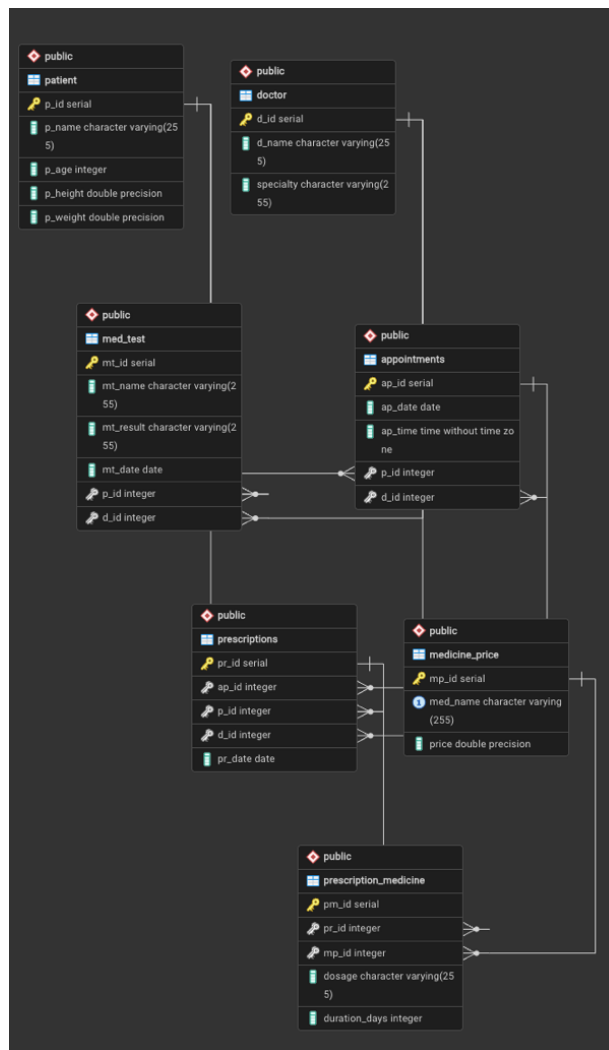
	pr_id integer	ap_id integer	p_id integer	d_id integer	pr_date date
> 1	9406	122	59		2026-01-10
> 2	2007	770	36		2026-04-19
> 3	8200	572	77		2023-02-23
> 4	7177	516	50		2025-05-09
> 5	2768	425	77		2024-04-07
> 6	4172	871	96		2026-02-26
> 7	3901	518	6		2026-01-03
> 8	2949	560	4		2026-08-09
> 9	3609	157	1		2023-05-15
> 10	5194	232	34		2023-06-25

- **prescription_medicine**: links prescriptions with medicines

A screenshot of a database query result for the 'pm' table. The table has columns: pm_id (integer), pr_id (integer), mp_id (integer), dosage (varchar(255)), and duration_days (integer). The results show 10 rows of data for various prescription-medicine links.

	pm_id integer	pr_id integer	mp_id integer	dosage varchar(255)	duration_days integer
> 1	8258	598		10 ml once daily	30
> 2	132	3414		1 tablet three times daily	3
> 3	1465	3146		1 capsule once daily	14
> 4	5408	1668		1 injection weekly	7
> 5	490	2386		Apply once daily	30
> 6	6365	1037		5 ml once daily	3
> 7	4596	4432		1 tablet once daily	10
> 8	1621	1211		2 tablets twice daily	7
> 9	2573	3304		Apply once daily	14
> 10	2397	4297		1 tablet three times daily	3

ERD Diagram:



Sample Queries:

```

SELECT * FROM patient;
SELECT * FROM doctor;
SELECT * FROM appointments;
SELECT * FROM med_test;
SELECT * FROM medicine_price;
SELECT * FROM prescriptions;
SELECT * FROM prescription_medicine;
  
```

```

SELECT p.p_name, d.d_name, a.ap_date, a.ap_time
FROM appointments a
JOIN patient p ON p.p_id = a.p_id
JOIN doctor d ON d.d_id = a.d_id
ORDER BY p.p_name;
  
```

```

SELECT * FROM get_doctor_appointments(1);
SELECT * FROM get_patient_medical_history(1);
SELECT * FROM get_patient_info(1);
  
```

```

-- Assuming there are records in med_test with mt_name = 'Blood Test' and mt_result = 'negative'
SELECT * FROM scan_reports('Blood Test', 'negative') ORDER BY mt_id;
-- Assuming pr_id = 1 exists in prescriptions table
SELECT * FROM get_prescription_details(1);
  
```

And a lot more queries added in https://github.com/karthik-bit1/CS377-A_hospital_medicaldb/blob/main/queries/queries.sql

Main Function Calls Added:

- Scan reports related to each requested scan and it's result.

```
SELECT * FROM scan_reports('Blood Test', 'negative') ORDER BY mt_id;
```

- Total number of appointments requested for doctor name.

```
SELECT * FROM get_total_appointments_of_doctor('Linda Harris');
```

- Prescriptions details of each prescription id requested

```
SELECT * FROM get_prescription_details(100);
```

- And a lot more other functions included in https://github.com/karthik-bit1/CS377-A_hospital_medicaldb/blob/main/queries/function.sql .

Main Procedure Calls Added:

- Inserting patient data into patient table

```
CALL insert_patient('Alice Smith', 30, 165.5, 60.0);
```

```
SELECT * FROM patient WHERE p_name = 'Alice Smith';
```

- Updating medical price in medicine_price table

```
CALL update_medicine_price(1, 19.99);
```

```
SELECT * FROM medicine_price WHERE mp_id = 1;
```

- Deleting existing appointments in appointments table

```
CALL delete_appointment(1);
```

```
SELECT * FROM appointments;
```

- And a lot more other Procedures included in https://github.com/karthik-bit1/CS377-A_hospital_medicaldb/blob/main/queries/procedures.sql .

Conclusion:

This project is used to manage data observation from different data tables included in the database. It is used widely in registrar office to manage bills, patients information and appointments, doctor computer to check their patients data and easy access in editing their prescriptions and appointments, and patients device to schedule appointments, gather information of medicines and check their reports.

Moving further, I will be adding an app.py to access data and have a direct report on search instead of prompting each SQL command line.